

Unit 4. Data Structure

4.1 Introduction of Data Structure and application areas.

4.2 Recursion concepts

4.3 Difference among Linear and Non-Linear Data Structure

4.4 Stack

- Concepts of Stack(LIFO)
- Pop, Push and Display(Peep)
- Application areas of Stack
(Infix to postfix, Infix to prefix)

4.1 Introduction of Data Structure and application areas.

A data structure is a storage system used to store and organise data. It arranges data on a computer in a way that allows for efficient access and updates.

Depending on your requirements and project, it is important to choose the right data structure. For example, if you want to store data sequentially in memory, you can use the array data structure.

The following are some of the most important areas where data structures are used:

Compiler design

Machine learning

Database design and management

Numerical and Statistical analysis

Operating system development

Types of Data Structures:

Data structures are basically divided into two categories:

1. Linear data structures
2. Non-linear data structures

Linear data structures

In linear data structures, the elements are arranged in sequence, one after the other. Since the elements are arranged in a particular order, they are easy to implement. However, as the complexity of the program increases, managing these structures can become challenging.

The linear data structures are:

- Arrays
- Queues
- Stacks
- Linked lists

Non linear data structures

Unlike linear data structures, elements in non-linear data structures are not arranged in a sequence. Instead, they are organised in a hierarchical manner where one element is connected to one or more elements.

The non-linear data structures are:

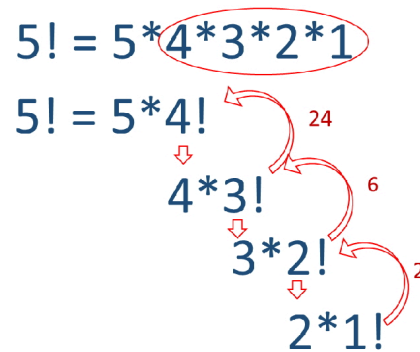
- Graphs
- Trees

4.2 Recursion concepts

This function calls itself repeatedly. It must have an exit condition that must be satisfied; otherwise, the recursive function will call itself indefinitely.

Recursive Algorithm for Factorial

1. **Base Case:**
 - a. If n is 1, return 1. ($1!=1$)
2. **Recursive Case:**
 - a. For any other positive integer n , return n multiplied by the factorial of $n-1$. (i.e. $n!=n \times \text{factorial}(n-1)!$)



Example 4.1 Write a program to find the factorial using a recursive function.

```
#include <iostream.h>
#include <conio.h>
class rec
{
    public:
        int fact(int);
};
int rec::fact(int n)//function definition
{
    if(n==1)
        return 1;
    else
        return n*fact(n-1);//recursive call
}

void main()
{
    rec r;
    int x=5;
    clrscr();
    cout<<"fact of "<<x<<" is "<<r.fact(x);//function call
    getch();
}
```

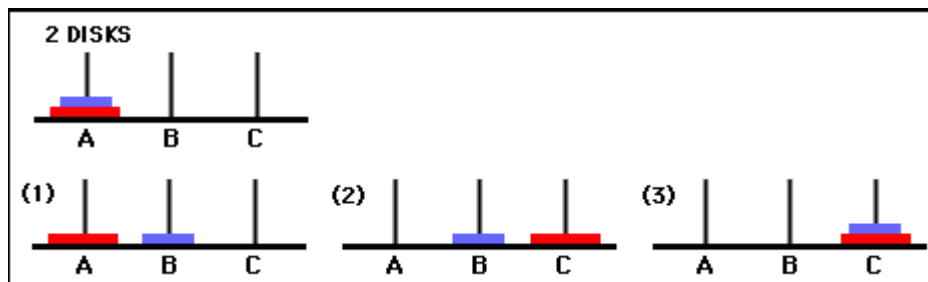
Output
fact of 5 is 120

Tower of Hanoi

The Tower of Hanoi is a well-known problem in computer science and can be solved using a recursive algorithm. It involves three rods and a number of disks of different sizes arranged in ascending order. The objective is to move the entire disk to another rod, following these rules:

1. Only one disk can be moved at a time.
2. Each move consists of taking the upper disk from one of the rod and placing it on top of another rod or on an empty rod.
3. No disk may be placed on top of a smaller disk.

Example with N=2 disk.



Algorithm

1. Base Case: If there is only one disk, move it from the source rod to the destination rod.

2. Recursive Case:

- Move the top (n-1) disks from the source rod to the auxiliary rod, using the destination rod as a temporary holding area.
- Move the nth disk (the largest disk) directly from the source rod to the destination rod.
- Move the (n-1) disks from the auxiliary rod to the destination rod, using the source rod as a temporary holding area.

Example 4.2 Tower of Hanoi implementation

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
void towerOfHanoi(int n, char a, char c, char b)
```

```
{
```

```
    if (n == 1)
```

```
    {
```

```
        cout<<" Move disk "<<n<<" from rod "<<a<<" to rod "<<c<<"\n";
```

```
        return;
```

```
    }
```

```

towerOfHanoi(n - 1,a,b,c);
cout << " Move disk " <<n<< " from rod " <<a<< " to rod " <<c<<"\n";
towerOfHanoi(n - 1,b,c,a);
}

int main()
{
    int N;
    clrscr();
    cout<<"\n enter n";
    cin>>N;
    // A, B and C are names of rods
    towerOfHanoi(N, 'A', 'C', 'B');
    getch();
    return 0;
}

```

Output :

```

enter n2
Move disk 1 from roa A to rod B
Move disk 2 from roa A to rod C
Move disk 1 from roa B to rod C

```

4.3 Difference among Linear and Non-Linear Data Structure

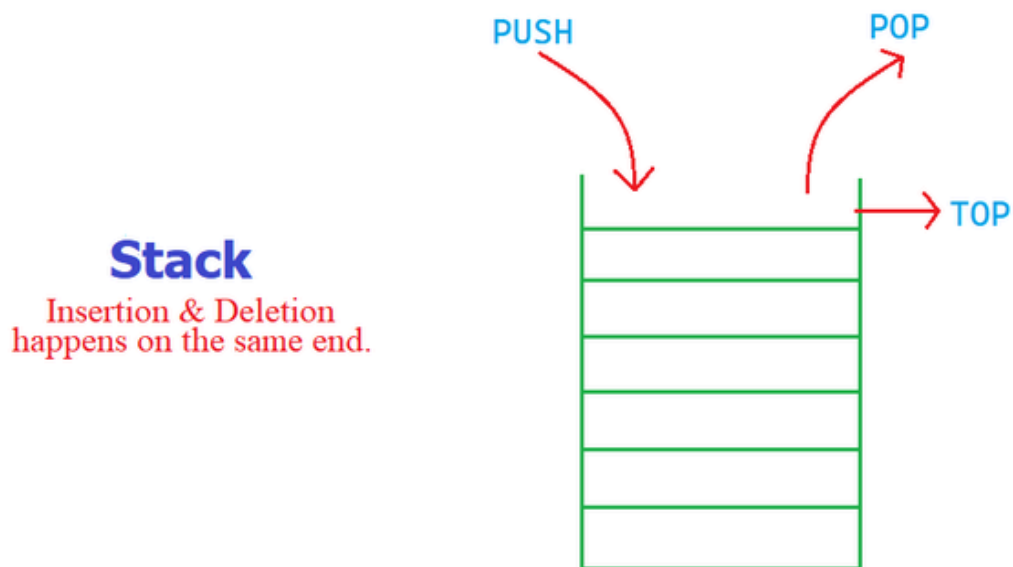
Linear Data Structures	Non Linear Data Structures
The data items are arranged in sequential order, one after the other.	The data items are arranged in non-sequential order (hierarchical manner).
All the items are present on the single layer.	The data items are present at different layers.
It can be traversed on a single run. That is, if we start from the first element, we can traverse all the elements sequentially in a single pass.	It requires multiple runs. That is, if we start from the first element it might not be possible to traverse all the elements in a single pass.
The memory utilisation is not efficient.	Different structures utilise memory in different efficient ways depending on the need.
The time complexity increases with the data size.	Time complexity remains the same.
Example: Arrays, Stack, Queue	Example: Tree, Graph, Map

4.4 Stack

- Concepts of Stack(LIFO)

- Pop, Push and Display(Peep)

A stack is a linear data structure that follows a particular order for performing operations. The order may be LIFO (Last In First Out) or FILO (First In Last Out).



There are many real-life examples of a stack. Consider the simple example of plates stacked on top of one another in a canteen. The plate on the top is the first one to be removed, i.e., the plate placed at the bottom remains in the stack for the longest period of time. This clearly demonstrates the LIFO/FILO order. The following four basic operations are mainly performed in a stack:

Stack Operations

The basic four operations of a stack are push, pop, peep, and change(UPDATE).

- 1) Push - Inserts an element X at the top of a stack represented by array S, which contains N elements, with a pointer TOP indicating the current top element of the stack.

Step 1: [check stack overflow]

 If $TOP \geq N$ then

 write 'stack overflow'

 return

Step 2: [Increment top]

$TOP = TOP + 1$

Step 3: [Insert element]

$S[TOP] = X$

Step 4: [finished]

 return

- 2) POP - removes the top element from a stack represented by the array S and returns that element. The pointer TOP indicates the current top element of the stack.

Step 1: [Check for underflow]

 If $TOP == 0$ then

 write 'stack underflow'

 return

Step 2: [Decrement TOP]

TOP = TOP -1

Step 3: [return element of stack]
return(S[TOP+1])

3) PEEP - return the value of the ith element from the top of the stack.

Step 1: [check for underflow]
If TOP-i+1 <=0 then
write ' stack underflow on peep'
return

Step 2: [return i th element from top of stack]
return(s[TOP-i+1])

4) CHANGE - Updates the value of the ith element from the top of the stack to the value stored in X.

Step 1: [check for underflow]
If TOP-i+1 <=0 then
write ' stack underflow on change'
return

Step 2: [change i th element from top of stack]
s[TOP-i+1]=X

Example 4.3 Stack implementation

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
class stack
{
int s[4];
int top;
public:
stack()
{top = 0;}
void push(int x)
{
if(top >= 3)
{
cout << "stack overflow";
return;
}
top=top+1;
s[top] = x;
cout << "inserted " << x;
}
void pop()
{
if(top == 0)
```

```

        {
            cout << "stack underflow";
            return;
        }
        cout << "deleted " << s[top--];
    }

void peep();
void change();
void display();
};

void stack::peep()
{
    int i,t;
    cout<<"enter the position";
    cin>>i;

    if(top-i+1<= 0)
    { cout << " stack underflow on peep"; return;}

    cout<<s[top-i+1];
}
void stack::change()
{
    int i,item;
    cout<<"\n enter the position to enter";
    cin>>i;
    cout<<"\n enter the item to be insert :";
    cin>>item;

    if(top-i+1<= 0)
    { cout << " stack underflow on change";return;}

    s[top-i+1]=item;
}

void stack::display()
{
    if(top == 0)
    {
        cout << " stack underflow";
        return;
    }
    for(int i = top; i >= 1; i--)
        cout <<"| s["<<i<<"] |"<< s[i] << "| \n";
}

int main()

```

```

{
int ch;
stack st;
clrscr();
while (1)
{
clrscr();
cout << "\n 1.push";
cout << "\n 2.pop";
cout << "\n 3.display";
cout << "\n 4.peep";
cout << "\n 5.change";
cout << "\n 6.exit";
cout << "\n enter ur choice: ";
cin >> ch;
switch (ch)
{
case 1:
cout << "enter the element: ";
cin >> ch;
st.push(ch);
break;
case 2:
st.pop();
break;
case 3:
st.display();
break;
case 4:
st.peep();
break;
case 5:
st.change();
break;
case 6:
exit(0);
}
getch();
}
}

```

Application areas of Stack

(Infix to postfix, Infix to prefix)

Infix to postfix

Infix expression: An expression of the form 'a op b' has an operator between every pair of operands. For example, 'a + b'.

Postfix expression: An expression of the form 'a b op' has an operator following every pair of operands. For example, 'ab+'.

Infix Expression : $A+(B*C+D)/E$

Push "(" into stack and add "(" to the end of the infix expression.

Input Token	Stack	Postfix Expression	Action
	(		Add '(' to the stack.
A	(	A	Add A to the postfix expression.
+	(+	A	Push '+' onto the stack.
(	(+(	A	Push '(' onto the stack.
B	(+(	AB	Add B to the postfix expression.
*	(+(*	AB	Push '*' onto the stack.
C	(+(*	ABC	Add C to the postfix expression.
+	(+(+	ABC*	'*' operator has higher precedence than '+', so pop * and add it to the postfix expression. Also, add '+' to the stack.
D	(+(+	ABC*D	Add D to the postfix expression.
)	(+	ABC*D+	')' has come so pop + and add it to the postfix expression. Also, remove '(' from the stack.
/	(+(/	ABC*D+	'+' has lower precedence than '/', so push '/' onto the stack.
E	(+(/	ABC*D+E	Add E to the postfix expression.
)		ABC*D+E/+	')' has come, so pop all operators one by one from the stack and add them to the postfix expression.

Algorithm

Step 1: Push "(" onto stack and add "(" to the end of the infix expression.

Step 2: Read the infix expression(left to right).Scan the characters one by one.

Step 3: Repeat until each character in the infix notation has been scanned.

If a "(" is encountered, push it onto the stack.

If an operand (whether a digit or a character) is encountered, add it to the postfix expression.

If a ")" is encountered, then

- Repeatedly pop operators from the stack and add them to the postfix expression until a "(" is encountered.
- Discard the "(" . Also remove the "(" from stack and do not add it to the postfix expression.

If an operator is encountered, then

- Repeatedly pop operators from the stack and add each operator (popped from the stack) to the postfix expression if it has the same precedence or a higher precedence than the current operator.
- Push the operator onto the stack.

[END OF IF]

[END OF LOOP]

Step 4: EXIT

Example 4.4 Infix to postfix expression

```
#include <iostream.h>
#include <ctype.h>
#include <conio.h>
#include <string.h>
#define SIZE 50

char stack[SIZE];
int top=-1;    /* Global declarations */
class postf
{
public:
void push(char elem)
{
    /* Function for PUSH operation */
    stack[++top]=elem;
}

char pop()
{
    /* Function for POP operation */
    return(stack[top--]);
}

int pr(char symbol)
{
    /* Function for precedence */

    if(symbol == '^')/* exponent operator, highest precedence*/
    {
        return(3);
    }
    else if(symbol == '*' || symbol == '/')
    {
        return(2);
    }
    else if(symbol == '+' || symbol == '-')    /* lowest precedence */
    {
        return(1);
    }
}
```

```

        else
        {
            return(0);
        }
    }
};
/* Main Program */
void main()
{
    char infix[50],postfix[50],ch;
    int i=0,k=0;
    postf p;
    clrscr();
    cout<<"Enter Infix Expression : ";
    cin>>infix;
    p.push('(');
    strcat(infix,"");
    while((ch=infix[i++]) != '\0')
    {
        if( ch == '(') p.push(ch);
        else if(isalnum(ch)) postfix[k++]=ch;
        else if( ch == ')')
        {
            while( stack[top] != '(')
                postfix[k++]=p.pop();
            p.pop(); /* Remove ( */
        }
        else
        { /* Operator */
            while(p.pr(stack[top]) >=p.pr(ch) )
                postfix[k++]=p.pop();
            p.push(ch);
        }
    }
    postfix[k]='\0'; /* Make postfix as valid string */
    cout<<"\n Postfix Expression = "<<postfix;
    getch();
}
output :
Enter Infix Expression :a+b
Postfix Expression = ab+

```

Infix to prefix

Infix expression: An expression of the form 'a op b' has an operator between every pair of operands. For example, 'a + b'.

Prefix expression: An expression of the form 'op a b' has an operator followed by operands. For example, +ab.

Infix Expression: (A+B)+C-(D-E)^F

Push “)” into the stack. then reverse the given infix expression(scan from right to left) and add “(” to the end of the infix expression.

After Reversing String is: F^E-D(-C+)B+A(

Input Token	Stack	Expression	Action
	)		Add ')' to the stack.
F	)	F	Add F to the expression string..
^	)^	F	Push '^' onto the stack.
)	)^)	F	Push ')' onto the stack.
E	)^)	FE	Add E to the expression string.
-	)^)-	FE	Push '-' onto the stack.
D	)^)-	FED	Add D to the expression string.
(	)^	FED-	'(' pair matched, so pop the operator '-' and add it to the expression string. Also, remove ')' from the stack.
-	)-	FED-^	'^' operator has higher precedence than '-', so pop '^' and add it to the expression string. Also, add the '-' operator to the stack.
C	)-	FED-^C	Add C to the expression string.
+	)+	FED-^C-	Pop '-' because '-' and '+' have the same precedence. Also, add the '+' operator to the stack.
)	)++)	FED-^C-	Push ')' onto the stack.
B	)++)	FED-^C-B	Add B to the expression string.
+	)++)+	FED-^C-B	Push '+' onto the stack.
A	)++)+	FED-^C-BA	Add A to the expression string.
(	)+	FED-^C-BA+	'(' pair matched, so pop the operator '+' and add it to the expression string. Also, remove ')' from the stack.
(	)+	FED-^C-BA++	'(' pair matched, so pop all operators one by one as we have reached the end of the expression.

Now **reverse** the expression to get prefix expression ++AB-C^--DEF

Algorithm

1. Push "(" onto the stack. Then, reverse the given infix expression(scan from right to left) and add "(" to the end of the infix expression.
2. Scan the characters one by one.
3. Repeat until each character in the infix notation has been scanned.
If the character is an operand, add it to the prefix expression.

If the character is a closing parenthesis, push it onto the stack.

If the character is an opening parenthesis

- pop the elements from the stack until we find the corresponding closing parenthesis.
- Discard the ")". Also remove the "(" from stack and do not add it to the prefix expression.

If the character scanned is an operator

- Repeatedly pop from stack and add each operator (popped from the stack) to the prefix expression if it has the same precedence or a higher precedence than the current operator.
- Push the operator onto the stack.

[END OF IF]

[END OF LOOP]

4. After all the characters are scanned, reverse the prefix notation.

Expression 4.5 Infix to prefix expression

```
#define SIZE 50          /* Size of Stack */
#include <iostream.h>
#include <conio.h>
#include <string.h>
#include <ctype.h>
char s[SIZE];
int top=-1; /* Global declarations */
class pref
{
public:
void push(char elem)
{
    /* Function for PUSH operation */
    s[++top]=elem;
}
char pop()
{
    /* Function for POP operation */
    return(s[top--]);
}
int pr(char symbol)
{
    /* Function for precedence */

    if(symbol == '^')/* exponent operator, highest precedence*/
    {
        return(3);
    }
}
```

```

        else if(symbol == '*' || symbol == '/')
        {
            return(2);
        }
        else if(symbol == '+' || symbol == '-')    /* lowest precedence */
        {
            return(1);
        }
        else
        {
            return(0);
        }
    }
};

void main()
{
    /* Main Program */
    char infix[50],prfx[50],ch;
    int i=0,k=0;
    pref p;
    clrscr();
    cout<<"\n\nInfix Expression";
    cin>>infix;
    p.push('(');
    strrev(infix);
    strcat(infix,"(");
    while( (ch=infix[i++]) != '\0')
    {
        if( ch == ')') p.push(ch);
        else if(isalnum(ch)) prfx[k++]=ch;
        else if( ch == '(')
        {
            while( s[top] != ')')
                prfx[k++]=p.pop();
            p.pop(); /* Remove ) */
        }
        else
        {
            /* Operator */
            while( p.pr(s[top]) >= p.pr(ch) )
                prfx[k++]=p.pop();
            p.push(ch);
        }
    }
    prfx[k]='\0';    /* Make prfx as valid string */
    strrev(prfx);
    cout<<"\n\n Prefix Exp:"<<prfx;
    getch();
}

```

Output

Infix expression a+b

Prefix Exp:+ab